

CourseHub - Technical Specification & Architecture Document

Table des Matières

1. Executive Summary & Architecture Overview

- 1.1 Executive Brief
- 1.2 Maturity Assessment
- 1.3 Technical Stack
- 1.4 Architectural Constraints
- 1.5 Critical Dependencies

2. Architecture Workflows & Visual Diagrams

3. Detailed Technical Specifications & Business Rules

- 3.1 Requirements Traceability
- 3.2 Security Rules
- 3.3 Data Models

4. Project Governance & Structural Gaps

- 4.1 Structural Gaps
- 4.2 Remediation & Workflow

5. Technical & Domain Glossary (Terminology Reference)

1. Executive Summary & Architecture Overview

1.1 Executive Brief

CourseHub is a course management platform API facilitating user authentication, course lifecycle management, and student enrollment tracking. The system implements a role-based access model (Student/Instructor) and manages the relational flow between Users, Courses, and Modules, with a specific focus on enrollment progress automation and strict deletion constraints to protect active learner data.

1.2 Maturity Assessment

The specification provides a high-quality technical contract with precise business rules and entity relationships. However, it lacks strategic context, specifically regarding overarching business goals and explicit scope boundaries, which could lead to implementation drift. Due to these structural gaps in the 'Why' and 'Out-of-Scope' definitions, the project status is REFINEMENT.

1.3 Technical Stack

- JWT
- Resend

1.4 Architectural Constraints

- Authentication: Mandatory JWT Bearer tokens in Authorization header.
- Response Format: Standardized envelope containing 'data', 'meta', and 'errors'.
- Data Validation: Enrollment progress must be an integer between 0 and 100 inclusive.
- Business Logic: Course deletion is forbidden if active enrollments exist (409 Conflict).
- Enrollment Logic: Students are restricted to published courses and unique enrollments per course.
- State Automation: `completed\_at` must be automatically set to current timestamp when progress reaches 100.

1.5 Critical Dependencies

- Resend: Required for asynchronous welcome email delivery upon registration.
- JWT: Essential for identity verification and session management.
- Enrollment -> Course: Strict dependency ensuring students only enroll in published courses.
- Course -> Enrollment: Referential integrity constraint preventing course deletion if active enrollments are present.
- Course -> Module: Structural containment relationship for course content.

2. Architecture Workflows & Visual Diagrams

(No diagrams provided in the source data)

3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

Identifier	Requirement Type	Description	Source Section
AUTH-JWT	Non-Functional	Authentication must use JWT Bearer tokens in the Authorization header.	API Overview
RESP-ENVELOPE	Non-Functional	All responses must follow a consistent envelope containing 'data', 'meta', and 'errors'.	Response Envelope
ENT-USER	Entity	User entity containing id, email, display_name, role, and created_at.	User
ENT-COURSE	Entity	Course entity containing title, description, price_cents, published status, and associated modules.	Course
ENT-MODULE	Entity	Module entity associated with a course with a specific order.	Module
ENT-ENROLLMENT	Entity	Enrollment entity linking a student to a course with progress tracking.	Enrollment
FR-REG-USER	Functional	Allow students to self-register with email, password, and display name.	POST <code>`/api/v1/auth/register`</code>
FR-REG-EMAIL	Functional	Send a welcome email asynchronously via Resend upon successful registration.	POST <code>`/api/v1/auth/register`</code>
FR-COURSE-CREATE	Functional	Allow instructors to create courses with modules.	POST <code>`/api/v1/courses`</code>
RULE-ACTIVE-ENROLL	Constraint	ActiveEnrollmentConstraint: A course cannot be deleted if it has active enrollments (returns 409 Conflict).	DELETE <code>`/api/v1/courses/{course_id}`</code>
FR-ENROLL-VALID	Functional	Students can only enroll in published courses and cannot enroll in the same course twice.	POST <code>`/api/v1/enrollments`</code>
FR-PROGRESS-AUTO	Functional	If enrollment progress is updated to 100 and completed_at is null, system must automatically set completed_at to current timestamp.	PUT <code>`/api/v1/enrollments/{enrollment_id}`</code>
RULE-PROGRESS-RANGE	Constraint	Enrollment progress must be an integer between 0 and 100 inclusive.	PUT <code>`/api/v1/enrollments/{enrollment_id}`</code>

3.2 Security Rules

- **Authentication**: All protected endpoints require a ``Authorization: Bearer `` header using JWT.
- **Authorization**: Role-based access control is enforced (e.g., ``FR-COURSE-CREATE`` is restricted to the Instructor role).

3.3 Data Models

- **User (ENT-USER)**: Core identity containing email, display name, and role.
- **Course (ENT-COURSE)**: Content container with pricing and publication status.
- **Module (ENT-MODULE)**: Ordered content units belonging to a Course.

- ****Enrollment (ENT-ENROLLMENT)****: Junction entity tracking the relationship and progress between a User and a Course.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Goals & Objectives	HIGH	The document defines 'How' (API) but not the overarching business 'Why' or high-level goals.
Scope & Out-of-Scope	MEDIUM	Define what the API does NOT handle (e.g., payment processing, content delivery) to avoid scope creep.
Open Questions & Uncertainties	LOW	Add a section for unresolved technical decisions (e.g., specific JWT expiration strategies or pagination limits).

4.2 Remediation & Workflow

The project is currently in the ****REFINEMENT**** phase. To move to a "Ready for Implementation" state, the product owner must provide the missing strategic context (Goals and Scope) to ensure the technical implementation aligns with business objectives.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
ActiveEnrollmentConstraint	BUSINESS_DOMAIN	RULE-ACTIVE-ENROLL	A validation rule that prevents the removal of a learning path if associated students are currently linked to it, triggering a 409 status code.
CORS Standard	TECHNICAL_STACK	RESP-ENVELOPE	The mechanism governing cross-origin resource sharing to ensure the API endpoints are accessible only by authorized external domains.
Cryptographic Hashing	TECHNICAL_STACK	FR-REG-USER	The process of transforming sensitive account credentials into a fixed-size string for secure persistence.
JWT	TECHNICAL_STACK	AUTH-JWT	The compact, URL-safe means of representing claims to be transferred between two parties, passed via the Bearer scheme in the request header.